

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 4 – Buổi 1

Khám phá và làm sạch dữ liệu
(Exploratory Data Analysis & Data Cleaning)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- Thực hiện các bước **khám phá dữ liệu** (*exploratory data analysis, EDA*): xem tổng quan, tìm ô trống, xem phân bố, so sánh nhóm bằng bảng và biểu đồ.
- Dùng **mốc so sánh** (*baseline*) để đánh giá một mô hình có thật sự hữu ích.
- **Làm sạch dữ liệu** (*data cleaning*): loại cột không phù hợp, xử lý **giá trị thiếu** (*missing values*), nhận diện **giá trị ngoại lai** (*outliers*).
- Chọn cách điền ô trống phù hợp: trung bình, trung vị, yếu vị, theo nhóm.

Nội dung

1. Bộ dữ liệu Titanic

2. Khám phá dữ liệu (EDA)

3. Làm sạch dữ liệu

4. Thực hành Buổi 4.1

5. Tổng kết Buổi 4.1

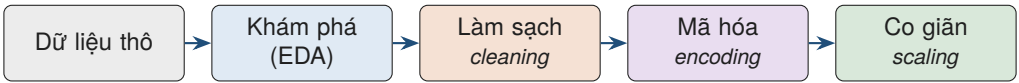
Đặt vấn đề: dữ liệu thật không bao giờ "sạch"

Iris, Wine rất "ngoan". Dữ liệu thực tế thường có:

- **Ô trống** (*missing values*): khách không khai tuổi.
- **Cột chữ** (*categorical*): giới tính, thành phố, hạng thành viên.
- **Thang đo chênh lệch** (*different scales*): tuổi 18–70, thu nhập 5–200 triệu.
- **Giá trị bất thường** (*outliers*): tuổi 250, đơn hàng 10 tỷ.

Định nghĩa: Tiền xử lý – Data preprocessing

Biến **dữ liệu thô** (*raw data*) thành dạng mà mô hình có thể học tốt. Thường tốn **nhiều công sức nhất** và quyết định chất lượng mô hình nhiều nhất.



Ví dụ đời thường: sơ chế trước khi nấu

Ví dụ đời thường: Không đầu bếp nào cho rau còn dính đất vào nồi

Rau mua từ chợ còn dính đất, có lá úa, có sâu; cá còn vảy. Phải nhặt, rửa, cắt, ướp – mất nhiều thời gian hơn cả lúc nấu. Dữ liệu cũng phải được "sơ chế" (*preprocessing*) trước khi đưa cho mô hình "nấu".

Buổi 4.1 (hôm nay)

Khám phá (*EDA*) và **làm sạch** (*cleaning*): ô trống, cột thừa, ngoại lai.

Buổi 4.2

Mã hóa (*encoding*), **co giãn** (*scaling*), quy trình chống **rò rỉ** (*leakage*).

Sơ chế ẩu thì đầu bếp giỏi đến đâu (thuật toán mạnh đến đâu) cũng không cứu được món ăn.

Phần 1

Bộ dữ liệu Titanic

Bộ dữ liệu Titanic

- Tàu chìm 15/4/1912, hơn 1 500 người thiệt mạng.
- 891 hành khách, 15 cột.
- **Bài toán**: dự đoán hành khách **sống sót** hay không – **phân loại nhị phân** (*binary classification*).
- Có đủ mọi vấn đề: ô trống, cột chữ, cột trùng ý nghĩa, ngoại lai.

Cột	Ý nghĩa	Kiểu
`survived`	**Nhãn**: 1 sống sót, 0 thiệt mạng	0/1
`pclass`	Hạng vé 1, 2, 3	thứ bậc
`sex`, `age`	Giới tính, tuổi	chữ, số
`sibsp`, `parch`	Số anh chị em/vợ chồng; cha mẹ/con đi cùng	số
`fare`, `embarked`	Giá vé; cảng lên tàu (C, Q, S)	số, chữ
`class`	Hạng vé dạng chữ	trùng `pclass`
`who`, `adult_male`	Đàn ông/phụ nữ/trẻ em	suy ra được
`deck`	Boong tàu	thiếu nhiều
`embark_town`	Tên cảng	trùng `embarked`
`alive`	yes/no	**trùng nhãn!**
`alone`	Đi một mình	True/False

Nạp dữ liệu

ch04.py — ô titanic_load

```python
titanic = sns.load_dataset("titanic")
print("Kích thước:", titanic.shape)
print(titanic.head())
```

Kết quả

```
Kích thước: (891, 15)
   survived  pclass     sex   age  sibsp  parch     fare embarked  class    who  \
0         0       3    male  22.0      1      0   7.2500        S  Third    man
1         1       1  female  38.0      1      0  71.2833        C  First  woman
2         1       3  female  26.0      0      0   7.9250        S  Third  woman
3         1       1  female  35.0      1      0  53.1000        S  First  woman
4         0       3    male  35.0      0      0   8.0500        S  Third    man

   adult_male deck  embark_town alive  alone
0        True  NaN  Southampton    no  False
1       False    C    Cherbourg   yes  False
2       False  NaN  Southampton   yes   True
3       False    C  Southampton   yes  False
4        True  NaN  Southampton    no   True
```

Bảng quá rộng nên pandas in thành hai khối (dấu `\` cuối dòng: bảng còn tiếp ở khối dưới). `NaN` (*Not a Number*) là **ô trống**.

Phần 2

Khám phá dữ liệu (EDA)

Ví dụ đời thường: bác sĩ khám trước khi kê đơn

Ví dụ đời thường: EDA là bước "khám bệnh" cho dữ liệu

Bác sĩ giỏi không kê thuốc ngay khi bệnh nhân bước vào: hỏi han, đo huyết áp, xem xét nghiệm, rồi mới quyết định. EDA là hỏi đúng câu hỏi, xem đúng chỉ số **trước khi** chọn "thuốc" (mô hình).

Định nghĩa: Khám phá dữ liệu – Exploratory Data Analysis

Dùng thống kê và biểu đồ để hiểu dữ liệu **trước khi** xây dựng mô hình: dữ liệu có gì, chất lượng ra sao, yếu tố nào có vẻ liên quan đến nhãn.

Bỏ qua bước này dễ "kê nhầm thuốc" – ví dụ đưa cột chứa đáp án vào mô hình mà không biết.

Xem cấu trúc và kiểu dữ liệu: `info()`

ch04.py — ô titanic_info

```
1  titanic.info()
```

Kết quả

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    category
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

Xem cấu trúc và kiểu dữ liệu: `info()`

ch04.py — ô titanic_info

```
1  titanic.info()
```

Kết quả

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    category
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

Câu hỏi cho lớp

Nhìn kết quả, bạn thấy những vấn đề gì?

Xem cấu trúc và kiểu dữ liệu: `info()`

ch04.py — ô titanic_info

```
1  titanic.info()
```

Kết quả

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    category
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

Câu hỏi cho lớp

Nhìn kết quả, bạn thấy những vấn đề gì?

1. `age` chỉ có 714/891 giá trị: **thiếu 177 ô**.
2. `deck` thiếu rất nhiều (chỉ 203 giá trị).
3. Nhiều cột `str`, `category`, `bool` cần **chuyển thành số**.

Cách đọc bảng `info()`

- Mỗi dòng của bảng là **một cột** của dữ liệu.
- `Non-Null Count`: số ô **có** giá trị. Nhỏ hơn tổng số dòng (891) $\Rightarrow$ cột có **ô trống** (*missing values*).
- `Dtype`: **kiểu dữ liệu** (*data type*) – số nguyên, số thực, chữ, `category`, `bool`.

Ba điều ghi nhận ngay

(1) `age` chỉ có 714/891 giá trị – thiếu 177 ô; (2) `deck` thiếu rất nhiều (chỉ 203 giá trị); (3) nhiều cột chữ, `category`, `bool` cần chuyển thành số.

Phân bố nhãn và mốc so sánh (*baseline*)

ch04.py — ô eda_nhan

```
print(titanic["survived"].value_counts())
print("Tỉ lệ sống sót:",
    round(titanic["survived"].mean(), 4))
```

Kết quả

```
survived
0    549
1    342
Name: count, dtype: int64
Tỉ lệ sống sót: 0.3838
```

- 342 người sống sót (38,4%), 549 thiệt mạng.

Lưu ý

Mốc so sánh (*baseline*): "mô hình" luôn đoán *thiệt mạng* (lớp đông nhất – *majority class*) đã đúng **61,6%**. Mọi mô hình ta xây dựng phải tốt hơn hẳn con số này mới có ý nghĩa.

Tìm ô trống

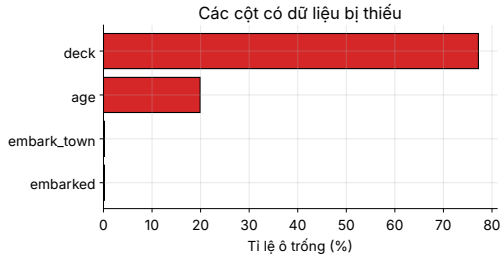
ch04.py — ô eda_thieu

```
so_thieu = titanic.isna().sum()
ti_le_thieu = (so_thieu / len(titanic) *
    ↪ 100).round(1)
bang_thieu = pd.DataFrame({"so_o_trong": so_thieu,
    ↪ "ti_le_%": ti_le_thieu})
print(bang_thieu[bang_thieu["so_o_trong"] > 0])
```

Kết quả

	so_o_trong	ti_le_%
age	177	19.9
embarked	2	0.2
deck	688	77.2
embark_town	2	0.2

`isna()`: bảng True/False (True ở ô trống); `.sum()`: đếm số True mỗi cột.



`deck` thiếu 77%, `age` gần 20%, `embarked` chỉ 2 ô. Mức độ thiếu quyết định cách xử lý (xóa cột, xóa dòng hay điền).

Thống kê các cột số

ch04.py — ô eda_describe

```
print(titanic[["age", "fare", "sibsp",
    "parch"]].describe().round(2))
```

Kết quả

	age	fare	sibsp	parch
count	714.00	891.00	891.00	891.00
mean	29.70	32.20	0.52	0.38
std	14.53	49.69	1.10	0.81
min	0.42	0.00	0.00	0.00
25%	20.12	7.91	0.00	0.00
50%	28.00	14.45	0.00	0.00
75%	38.00	31.00	1.00	0.00
max	80.00	512.33	8.00	6.00

- Tuổi trung bình ≈ 30; trẻ nhất 0,42 tuổi (5 tháng).
- `fare`: trung bình 32,2 nhưng trung vị chỉ 14,45, lớn nhất 512,33.

Mẹo nhỏ

Trung bình **lớn hơn nhiều** trung vị $\Rightarrow$ dữ liệu **lệch phải** (*right-skewed*): đa số vé rẻ, một số ít vé rất đắt – nghĩ ngay đến **ngoại lai** (Chương 3).

So sánh tỉ lệ sống sót giữa các nhóm

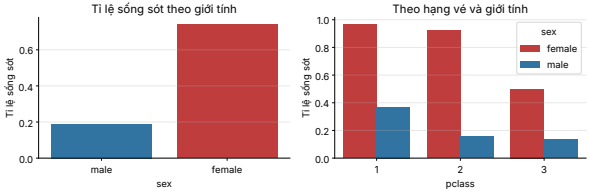
ch04.py — ô eda_nhom

```
print(titanic.groupby("sex")["survived"].mean().round(3))
print(titanic.groupby("pclass")["survived"].mean().round(3))
```

Kết quả

```
sex
female    0.742
male      0.189
Name: survived, dtype: float64
pclass
1    0.630
2    0.473
3    0.242
Name: survived, dtype: float64
```

Nhãn là 0/1 ⇒ **trung bình trong một nhóm = tỉ lệ sống sót** (*survival rate*) của nhóm đó.



- Nữ 74% – nam 19%: “phụ nữ và trẻ em trước”.
- Hạng 1 sống sót gấp 2,6 lần hạng 3.

Bài tập mẫu: tỉ lệ sống sót là trung bình của cột 0/1

Tám hành khách (giới tính, sống sót): (nữ,1), (nam,0), (nam,0), (nữ,1), (nam,1), (nam,0), (nữ,0), (nam,0).

Bài tập mẫu

Lời giải.

- Nữ: nhãn 1, 1, 0 $\Rightarrow$ 2 người sống, tỉ lệ $2/3 \approx 0{,}667 = (1+1+0)/3$.
- Nam: 0, 0, 1, 0, 0 $\Rightarrow$ tỉ lệ $1/5 = 0{,}2 =$ trung bình.

Cộng các số 0/1 = **đếm số 1**, nên trung bình cột 0/1 = tỉ lệ số 1.

ch04.py — ô eda_nho

```
# Ví dụ nhỏ để tính tay: 8 hành khách
nho = pd.DataFrame({"sex": ["female", "male", "male", "female",
    ↪  "male", "male", "female", "male"],
                    "survived": [1, 0, 0, 1, 1, 0, 0, 0]})
print(nho.groupby("sex")["survived"].agg(["count", "sum", "mean"]))
```

Kết quả

```
        count  sum      mean
sex
female      3    2  0.666667
male        5    1  0.200000
```

`count`: số người; `sum`: số sống sót; `mean`: tỉ lệ.

Bảng chéo `pd.crosstab`

ch04.py — ô eda_crosstab

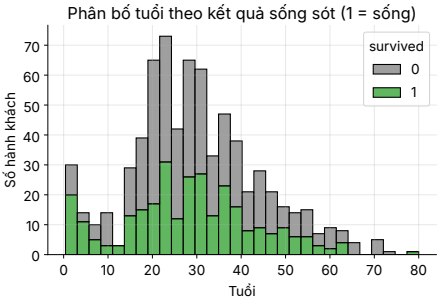
```
# Bảng chéo: mỗi dòng là một giới tính, chia theo kết quả; normalize="index" -> tỉ lệ theo dòng
print(pd.crosstab(titanic["sex"], titanic["survived"]))
print(pd.crosstab(titanic["sex"], titanic["survived"], normalize="index").round(3))
```

Kết quả

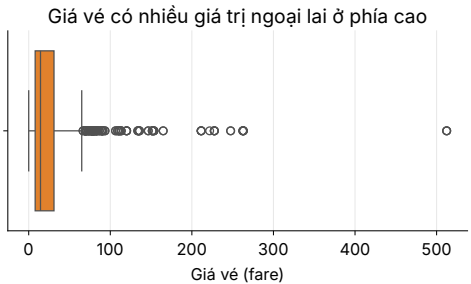
```
survived    0    1
sex
female     81  233
male      468  109
survived      0      1
sex
female    0.258  0.742
male      0.811  0.189
```

- **Bảng chéo** (*contingency table*): đếm số hành khách theo từng cặp (giới tính, kết quả).
- `normalize="index"`: đổi số đếm thành tỉ lệ **trên mỗi dòng**.
- Phụ nữ: $233/(81+233) = 0{,}742$ – trùng kết quả `groupby`.

Tuổi và giá vé



Màu xanh (sống sót) chiếm tỉ lệ cao ở trẻ em dưới 10 tuổi.



Giá vé có nhiều ngoại lai ở phía cao.

Mẹo nhỏ

EDA tối thiểu – **5 câu hỏi**: (1) bảng bao nhiêu dòng, cột, kiểu gì? (2) nhãn phân bố thế nào? (3) cột nào có ô trống? (4) cột số có bất thường không? (5) cột nào liên quan đến nhãn?

Năm câu hỏi EDA tối thiểu – và tự kiểm tra

1. Bảng có bao nhiêu dòng, cột, kiểu gì?
2. Nhãn phân bố thế nào?
3. Cột nào có ô trống, bao nhiêu?
4. Các cột số có giá trị bất thường không?
5. Những cột nào có vẻ liên quan đến nhãn?

Tự kiểm tra nhanh

(1) Cột có `Non-Null Count` 600 trong bảng 750 dòng thiếu bao nhiêu %? (2) Vì sao "luôn đoán thiệt mạng" đạt 61,6%? (3) Trung bình `survived` của hạng 1 là 0,63 nghĩa là gì?

Năm câu hỏi EDA tối thiểu – và tự kiểm tra

1. Bảng có bao nhiêu dòng, cột, kiểu gì?
2. Nhãn phân bố thế nào?
3. Cột nào có ô trống, bao nhiêu?
4. Các cột số có giá trị bất thường không?
5. Những cột nào có vẻ liên quan đến nhãn?

Tự kiểm tra nhanh

(1) Cột có `Non-Null Count` 600 trong bảng 750 dòng thiếu bao nhiêu %? (2) Vì sao "luôn đoán thiệt mạng" đạt 61,6%? (3) Trung bình `survived` của hạng 1 là 0,63 nghĩa là gì?

Đáp án: (1) 20%. (2) Vì 61,6% hành khách thật sự thiệt mạng. (3) 63% hành khách hạng 1 sống sót.

Phần 3

Làm sạch dữ liệu

Loại bỏ cột không phù hợp

ch04.py — ô lam_sach_cot

```
print(pd.crosstab(titanic["survived"],
    ↪ titanic["alive"]))
cot_bo = ["alive", "class", "who", "adult_male",
    ↪ "embark_town", "deck"]
df = titanic.drop(columns=cot_bo)
print("Còn lại", df.shape[1], "cột:",
    ↪ df.columns.tolist())
```

Kết quả

```
alive      no  yes
survived
0         549    0
1           0  342
Còn lại 9 cột: ['survived', 'pclass', 'sex', 'age',
↪ 'sibsp', 'parch', 'fare', 'embarked', 'alone']
```

`alive = yes` **đúng hoàn toàn** khi `survived = 1` ⇒ mô hình "nhìn trộm" đáp án, accuracy 100% vô nghĩa.

Loại cột vì 3 lý do:

- **Chứa đáp án**: `alive` – trường hợp nghiêm trọng nhất của **rò rỉ dữ liệu** (*data leakage*).
- **Trùng thông tin** (*redundant*): `class`, `embark_town`, `who`, `adult_male`.
- **Thiếu quá nhiều**: `deck` (77%).

Dòng trùng lặp (*duplicates*): có nên xóa?

ch04.py — ô lam_sach_trung

```
print("Số dòng trùng lặp:", df.duplicated().sum())
```

Kết quả

```
Số dòng trùng lặp: 111
```

Dòng trùng lặp (*duplicates*): có nên xóa?

ch04.py — ô lam_sach_trung

```
print("Số dòng trùng lặp:", df.duplicated().sum())
```

Kết quả

```
Số dòng trùng lặp: 111
```

Câu hỏi cho lớp

Có 111 dòng trùng. Có nên xóa không?

Dòng trùng lặp (*duplicates*): có nên xóa?

ch04.py — ô lam_sach_trung

```
print("Số dòng trùng lặp:", df.duplicated().sum())
```

Kết quả

```
Số dòng trùng lặp: 111
```

Câu hỏi cho lớp

Có 111 dòng trùng. Có nên xóa không?

Lưu ý

Không nên xóa vội. Dữ liệu không có tên hành khách: hai người khác nhau cùng giới tính, hạng vé, giá vé, cùng thiếu tuổi… trông giống hệt nhau. Chỉ xóa khi chắc chắn là lỗi ghi hai lần (ví dụ trùng mã đơn hàng).

Các chiến lược xử lý giá trị thiếu (*missing values*)

Chiến lược	Khi nào dùng	Nhược điểm
Xóa dòng *(drop rows)*	Rất ít dòng bị thiếu (dưới khoảng 5%)	Mất dữ liệu; có thể làm lệch mẫu
Xóa cột *(drop column)*	Thiếu quá nhiều (trên 50–60%) hoặc không quan trọng	Mất một đặc trưng
Điền trung bình / trung vị *(mean / median imputation)*	Cột số; **trung vị** an toàn hơn khi có ngoại lai	Giảm độ phân tán của cột
Điền yếu vị *(mode)*	Cột phân loại (chữ)	Nhóm phổ biến càng phổ biến
Điền theo nhóm *(group-wise)*	Giá trị thiếu phụ thuộc cột khác (tuổi theo hạng vé, giới tính)	Phức tạp hơn

Bài tập mẫu: điền trung bình hay trung vị?

Lương (triệu đồng) 7 nhân viên: 8, 9, *trống*, 10, 11, *trống*, 95. Điền hai ô trống thế nào?

Bài tập mẫu

Lời giải. Năm giá trị có sẵn: 8, 9, 10, 11, 95.

- Trung bình $= 133/5 = 26{,}6$ – mức lương **không ai** trong 4 người bình thường có.
- Trung vị $= 10$ – rất "điển hình".

95 (lương giám đốc) kéo trung bình lên $\Rightarrow$ **điền trung vị an toàn hơn**.

ch04.py — ô thieu_nho

```
1  # Ví dụ nhỏ: điền ô trống bằng trung bình hay trung vị?
2  luong = pd.Series([8, 9, None, 10, 11, None, 95])   # triệu đồng; có 1
   ↪ giá trị rất lớn
3  print("Trung bình:", round(luong.mean(), 2), "  Trung vị:",
   ↪ luong.median())
4  print("Điền trung bình:",
   ↪ luong.fillna(luong.mean()).round(1).tolist())
5  print("Điền trung vị  :", luong.fillna(luong.median()).tolist())
```

Kết quả

```
Trung bình: 26.6   Trung vị: 10.0
Điền trung bình: [8.0, 9.0, 26.6, 10.0, 11.0, 26.6, 95.0]
Điền trung vị  : [8.0, 9.0, 10.0, 10.0, 11.0, 10.0, 95.0]
```

`fillna(giá_trị)`: thay mọi ô trống của cột bằng `giá_trị`. Ô trống trong pandas hiện là `NaN` hoặc `None`; hầu hết mô hình `scikit-learn` **báo lỗi** khi gặp ô trống.

Xóa dòng hay điền giá trị *(imputation)*?

ch04.py — ô thieu_xoa

```
print("Số dòng ban đầu        :", len(df))
print("Xóa mọi dòng có ô trống:",
    ↪ len(df.dropna()))
```

Kết quả

```
Số dòng ban đầu        : 891
Xóa mọi dòng có ô trống: 712
```

Mất 179 hành khách (20%) – **quá lãng phí**.

ch04.py — ô thieu_dien

```
df = df.copy()
trung_vi_tuoi = df["age"].median()
yeu_vi_cang = df["embarked"].mode()[0]
print("Trung vị tuổi:", trung_vi_tuoi, " Cảng phổ
    ↪ biến nhất:", yeu_vi_cang)
df["age"] = df["age"].fillna(trung_vi_tuoi)
df["embarked"] =
    ↪ df["embarked"].fillna(yeu_vi_cang)
print("Số ô trống còn lại:",
    ↪ df.isna().sum().sum())
```

Kết quả

```
Trung vị tuổi: 28.0  Cảng phổ biến nhất: S
Số ô trống còn lại: 0
```

- `df.copy()`: bản sao độc lập, tránh sửa vô tình bảng gốc.
- `.mode()` trả về Series (có thể có nhiều yếu vị) ⇒ lấy `[0]`.

Điền theo nhóm cho chính xác hơn

ch04.py — ô thieu_theo_nhom

```
goc = titanic.copy()
tuoi_theo_nhom = goc.groupby(["pclass",
    ↪  "sex"])["age"].median()
print(tuoi_theo_nhom)
```

Kết quả

```
pclass  sex
1       female    35.0
        male      40.0
2       female    28.0
        male      30.0
3       female    21.5
        male      25.0
Name: age, dtype: float64
```

Ví dụ

Hành khách hạng 1 thường lớn tuổi hơn hạng 3. Một nam giới hạng 1 bị thiếu tuổi được điền **40** thay vì 28 – sát thực tế hơn.

Thực hiện ở Bài tập 4.5 với `groupby(...).transform("median")`.

Giá trị ngoại lai

ch04.py — ô ngoai_lai

```
q1, q3 = df["fare"].quantile([0.25, 0.75])
iqr = q3 - q1
can_tren = q3 + 1.5 * iqr
print(f"Q1 = {q1:.2f}, Q3 = {q3:.2f}, IQR = {iqr:.2f},
    ↪ cận trên = {can_tren:.2f}")
print("Số vé ngoại lai:", (df["fare"] > can_tren).sum())
```

Kết quả

```
Q1 = 7.91, Q3 = 31.00, IQR = 23.09, cận trên = 65.63
Số vé ngoại lai: 116
```

Quy tắc 1,5 IQR (Chương 3): 116 vé đắt hơn 65,63 bảng là **ngoại lai** (*outlier*) – nhưng **ngoại lai** $\neq$ **sai**: đó là vé hạng sang có thật.

1. **Xóa**: chỉ khi chắc chắn là lỗi (tuổi 250).
2. **Giữ nguyên**: giá trị thật, mô hình ít bị ảnh hưởng.
3. **Biến đổi** (*transform*): ví dụ lấy logarit.

Logarit “nén” các số lớn thế nào?

ch04.py — ô log_tay

```
gia = np.array([0, 9, 99, 999])
print("log1p:", np.log1p(gia).round(3))     # log(1 + x)
```

Kết quả

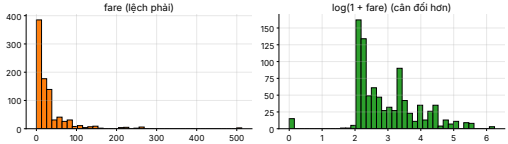
```
log1p: [0.    2.303 4.605 6.908]
```

- Trên thang gốc: 9 và 999 cách nhau 990. Trên thang log: chỉ cách nhau $6{,}908 - 2{,}303 \approx 4{,}6$.
- Giá trị rất lớn bị nén mạnh, giá trị nhỏ ít bị ảnh hưởng.
- `np.log1p(x)` $= \log(1 + x)$: tránh lỗi $\log(0)$ với vé giá 0.

Biến đổi logarit (*log transform*)

ch04.py — ô log_fare

```
log_fare = np.log1p(df["fare"])          # log(1 + fare), tránh log(0)
fig, ax = plt.subplots(1, 2, figsize=(11, 3.4))
ax[0].hist(df["fare"], bins=40, color="tab:orange", edgecolor="k")
ax[0].set_title("fare (lệch phải)")
ax[1].hist(log_fare, bins=40, color="tab:green", edgecolor="k")
ax[1].set_title("log(1 + fare) (cân đối hơn)")
plt.tight_layout()
plt.show()
```



Toán dễ hiểu: Vì sao log làm phân bố cân đối hơn?

$\log 10 \approx 2{,}3$; $\log 100 \approx 4{,}6$; $\log 1000 \approx 6{,}9$. Từ 10 đến 1000: thang gốc cách 990, thang log chỉ cách 4,6 $\Rightarrow$ giá trị lớn bị "nén" mạnh. `np.log1p(x)` $= \log(1+x)$ tránh lỗi $\log(0)$.

Tự kiểm tra nhanh: làm sạch dữ liệu

1. Cột `deck` thiếu 77%. Nên điền hay xóa cột? Vì sao?
2. Dãy 3, 5, *trống*, 7, 100: điền ô trống bằng trung vị thì được bao nhiêu?
3. Dữ liệu y tế ghi một bệnh nhân có nhiệt độ 3,7 độ C. Giữ, sửa hay xóa?

Tự kiểm tra nhanh: làm sạch dữ liệu

1. Cột `deck` thiếu 77%. Nên điền hay xóa cột? Vì sao?
2. Dãy 3, 5, *trống*, 7, 100: điền ô trống bằng trung vị thì được bao nhiêu?
3. Dữ liệu y tế ghi một bệnh nhân có nhiệt độ 3,7 độ C. Giữ, sửa hay xóa?

Đáp án

(1) Xóa cột – điền 77% gần như đoán mò. (2) Trung vị của 3, 5, 7, 100 là 6. (3) Gần như chắc chắn là lỗi nhập liệu (37 thành 3,7): sửa nếu xác minh được, nếu không coi như ô trống.

Phần 4

Thực hành Buổi 4.1

Thực hành 4.1: Khám phá dữ liệu Penguins – Bước 1

344 chim cánh cụt, 3 loài (Adelie, Chinstrap, Gentoo): chiều dài, độ dày mỏ (mm), chiều dài vây (mm), cân nặng (g), đảo, giới tính.

ch04.py — ô lab_buoc1

```
peng = sns.load_dataset("penguins")
print(peng.shape)
print(peng.isna().sum())
print(peng["species"].value_counts())
```

Kết quả

```
(344, 7)
species              0
island               0
bill_length_mm       2
bill_depth_mm        2
flipper_length_mm    2
body_mass_g          2
sex                 11
dtype: int64
species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64
```

Cột số thiếu 2 ô, `sex` thiếu 11 ô; ba loài không cân bằng – Chinstrap chỉ 68 con.

Thực hành 4.1: Bước 2 – số đo trung bình mỗi loài

ch04.py — ô lab1_buoc2

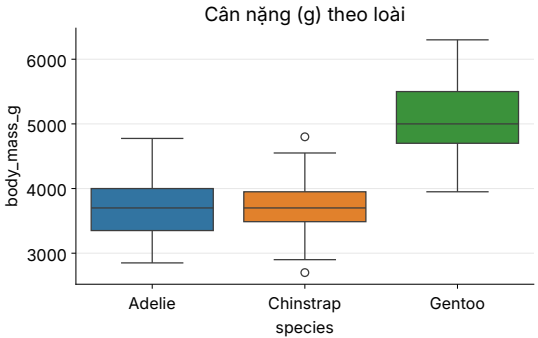
```
print(peng.groupby("species")[["bill_length_mm", "bill_depth_mm",
                               "flipper_length_mm", "body_mass_g"]].mean().round(1))
```

Kết quả

species	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
Adelie	38.8	18.3	190.0	3700.7
Chinstrap	48.8	18.4	195.8	3733.1
Gentoo	47.5	15.0	217.2	5076.0

- Gentoo nặng hơn hẳn ($\approx 5\,076$ g), vây dài nhất.
- Adelie mỏ ngắn nhất (38,8 mm); Chinstrap và Gentoo mỏ dài tương đương nhưng mỏ Gentoo **mỏng** hơn.
- Mỗi loài có “dấu hiệu” riêng $\Rightarrow$ bài toán phân loại có vẻ không quá khó.

Thực hành 4.1: Bước 3 – cân nặng theo loài



ch04.py — ô lab1_buoc3

```
1  plt.figure(figsize=(6.4, 3.6))
2  sns.boxplot(data=peng, x="species", y="body_mass_g",
   ↪  hue="species")
3  plt.title("Cân nặng (g) theo loài")
4  plt.show()
```

- Hộp Gentoo nằm hẳn phía trên: chỉ cần cân nặng đã tách được Gentoo.
- Adelie và Chinstrap chồng lên nhau ⇒ cần đặc trưng khác (chiều dài mỏ).

Thực hành 4.1: Bước 4 – điền ô trống

ch04.py — ô lab1_buoc4

```
peng_sach = peng.copy()
for cot in ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]:
    peng_sach[cot] = peng_sach[cot].fillna(peng_sach[cot].median())
peng_sach["sex"] = peng_sach["sex"].fillna(peng_sach["sex"].mode()[0])
print("Số ô trống sau khi điền:", peng_sach.isna().sum().sum())
print(peng[peng["sex"].isna()].head(3))
```

Kết quả

```
Số ô trống sau khi điền: 0
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
3  Adelie  Torgersen             NaN            NaN                NaN
8  Adelie  Torgersen            34.1           18.1              193.0
9  Adelie  Torgersen            42.0           20.2              190.0

   body_mass_g  sex
3          NaN  NaN
8       3475.0  NaN
9       4250.0  NaN
```

Cột số điền trung vị, `sex` điền yếu vị.

Lưu ý

Điền trên **toàn bộ** dữ liệu chỉ để luyện tay. Buổi 4.2 sẽ giải thích vì sao khi xây dựng mô hình, việc điền phải làm **sau** khi chia tập.

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Khám phá dữ liệu	Exploratory data analysis (EDA)
Làm sạch dữ liệu	Data cleaning
Tiền xử lý	Preprocessing
Kiểu dữ liệu	Data type
Ô trống, giá trị thiếu	Missing value (NaN)
Mốc so sánh	Baseline
Bảng chéo	Contingency table (crosstab)
Phân bố	Distribution

Tiếng Việt	Tiếng Anh
Lệch phải	Right-skewed
Dòng trùng lặp	Duplicate row
Điền giá trị thiếu	Imputation
Yếu vị	Mode
Giá trị ngoại lai	Outlier
Khoảng tứ phân vị	Interquartile range (IQR)
Biến đổi logarit	Log transform
Rò rỉ dữ liệu	Data leakage

Phần 5

Tổng kết Buổi 4.1

Tóm tắt Buổi 4.1

- **EDA** trả lời 5 câu hỏi: cấu trúc bảng, phân bố nhãn, ô trống, giá trị bất thường, cột nào liên quan đến nhãn. Công cụ: `info`, `describe`, `isna().sum()`, `groupby`, `crosstab`, histogram, boxplot.
- Trung bình cột nhãn 0/1 trong một nhóm = tỉ lệ của nhóm; tỉ lệ lớp đông nhất là **mốc so sánh** (*baseline*).
- **Làm sạch**: loại cột chứa đáp án, trùng thông tin hoặc thiếu quá nhiều; điền ô trống bằng trung vị (số), yếu vị (chữ) hoặc theo nhóm.
- Ngoại lai không có nghĩa là sai: chỉ xóa khi chắc chắn là lỗi; có thể dùng logarit để giảm ảnh hưởng.

Câu hỏi thảo luận

1. Nêu năm câu hỏi cần trả lời khi khám phá một bộ dữ liệu mới.
2. Vì sao không được đưa cột `alive` vào mô hình Titanic? Nêu ví dụ tương tự cho bài toán dự đoán sinh viên bỏ học.
3. Khi nào nên xóa dòng có ô trống, khi nào nên điền? Vì sao điền trung vị an toàn hơn trung bình?
4. Vì sao không nên xóa vội các dòng trùng lặp trong Titanic?

Bài tập Buổi 4.1

- ⋆ **4.1**: Tỉ lệ sống sót theo `embarked` và theo `alone`; vẽ biểu đồ cột, nêu nhận xét.
- ⋆ **4.2**: Cột `nhom_tuoi` (trẻ em, thiếu niên, người lớn, cao tuổi) bằng `pd.cut`; tỉ lệ sống sót mỗi nhóm.
- ⋆⋆ **4.3**: Điền tuổi theo trung vị nhóm (`pclass`, `sex`) bằng `groupby(...).transform("median")`; so sánh histogram.
- ⋆ **4.4**: Cân nặng 52, 55, *trống*, 58, 60, 61, 140, *trống*: tính tay giá trị điền và ngoại lai (IQR).
- ⋆⋆ **4.5**: Penguins: điền cân nặng theo trung vị **từng loài**; so với trung vị chung.

Chi tiết: *Bài tập Buổi 4.1*, Chương 4. Buổi sau: **Buổi 4.2 – Mã hóa, co giãn và quy trình chống rò rỉ**.